If the same failure happens, the change to the `products` table won't be made. This makes this code idempotent in a certain context—it can be safely retried until the last line executes, when the transaction is said to have *committed*. Once the transaction commits, if the entire thing is retried, `quantity_remaining` would be too low and the value for `charge_completed_at` would be incorrect.

Despite this subtlety, it's still a good idea to wrap multiple calls to the database inside a transaction. As you can see, it does eliminate certain problems from happening. There are some caveats, however.

When you open a transaction, the database creates locks on the tables or rows being updated (depending the specifics of what is being changed). This can cause other jobs and requests to block or even timeout, triggering your error catcher. If this happens a lot, you can have a cascading failure across the entire system.

A common cause of this problem that many developers think is clever (myself previously included) is to put code to queue a job inside the transaction block. The thinking is that if there is a problem communicating with Redis or otherwise queuing the job, it will trigger a rollback of any database changes, making the code safe to retry.

```
ActiveRecord::Base.transaction do
  product.update!(quantity_remaining: product.quantity_remaining - 1)
  order.update!(charge_id: payments_response.charge_id,
                charge_completed_at: Time.zone.now,
                charge_successful: true)
»   SendOrderNotificationEmailJob.perform_async(order.id)
end
```

This will hold all the locks created by `ActiveRecord::Base.transaction` open while Sidekiq communicates with Redis. This might not seem like a long time, but it can sometimes be *very* long, especially if you are in a cloud-hosted environment. The reason I no longer do this—and strongly